

returning a `Double`. What if `mean` is called as part of a larger computation and we'd like to abort that computation if `mean` is undefined? Or perhaps we'd like to take some completely different branch in the larger computation in this case? Simply passing an `onEmpty` parameter doesn't give us this freedom.

We need a way to defer the decision of how to handle undefined cases so that they can be dealt with at the most appropriate level.

4.3 The Option data type

The solution is to represent explicitly in the return type that a function may not always have an answer. We can think of this as deferring to the caller for the error-handling strategy. We introduce a new type, `Option`. As we mentioned earlier, this type also exists in the Scala standard library, but we're re-creating it here for pedagogical purposes:

```
sealed trait Option[+A]
case class Some[+A](get: A) extends Option[A]
case object None extends Option[Nothing]
```

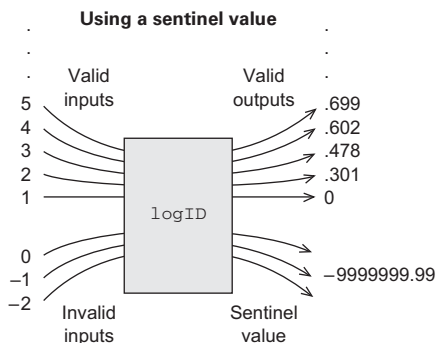
`Option` has two cases: it can be defined, in which case it will be a `Some`, or it can be undefined, in which case it will be `None`.

We can use `Option` for our definition of `mean` like so:

```
def mean(xs: Seq[Double]): Option[Double] =
  if (xs.isEmpty) None
  else Some(xs.sum / xs.length)
```

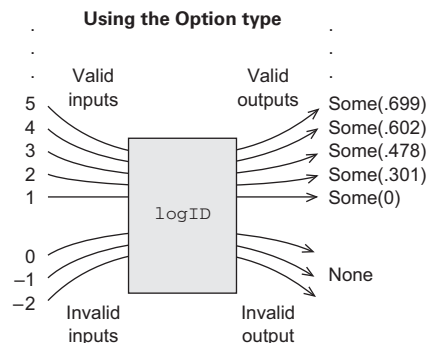
The return type now reflects the possibility that the result may not always be defined. We still always return a result of the declared type (now `Option[Double]`) from our function, so `mean` is now a *total function*. It takes each value of the input type to exactly one value of the output type.

Responding to invalid inputs



```
logID: Double => Double
```

Mapping all invalid inputs to a special value of the same type as the valid outputs. Ambiguous, and compiler can't check that caller handles it correctly.



```
logID: Double => Option[Double]
```

Every valid output is wrapped in `Some`. Invalid inputs are mapped to `None`. The compiler forces the caller to deal explicitly with the possibility of failure.